



Faculté des Sciences du vivant

Année universitaire 2026-2027

Rapport de Projet court

ALIGNEMENT MULTIPLE PROGRESSIF (MÉTHODE CLUSTAL)

*Implémentation par programmation dynamique exacte et
embranchement séquentiel*

Dépôt GitHub du projet : <https://github.com/Linamagroun/clustal>

Présenté par : MAGROUN Lina

UE: Gestion de projet et Programmation avancée

Formation : M2 Bioinformatique

Responsables : Y. Mohseni, K. Bouyioukos, J.C.Gelly, P. Fuchs

Sommaire

1. Introduction	1
2. Matériel et Méthodes	1
2.1 Jeu de données et détection du type moléculaire	1
2.2 Calcul des distances et pénalités de substitution	1
2.3 Alignement par paires : l'algorithme de Gotoh	2
2.4 Matrice de similarité et arbre glouton	2
2.5 Alignement progressif et gestion des flottants	2
2.6 Évaluation des performances algorithmiques	3
3. Résultats	3
3.1 Construction de l'arbre guide et ordre d'intégration	3
3.2 Analyse de l'alignement multiple final	4
3.3 Mesures de performances (Benchmark)	4
4. Discussion	5
5. Conclusion	5
6. Références	5

1. Introduction

L'alignement multiple de séquences constitue une méthode de base pour identifier des régions conservées au cours de l'évolution. Bien que l'algorithme de Needleman et Wunsch (1970) fournisse un alignement optimal pour deux séquences, son application exacte à N séquences exige le calcul d'une matrice multidimensionnelle. L'espace mémoire et le temps de traitement croissent alors de manière exponentielle, rendant le calcul intraitable sur des données réelles.

En réponse à cette limitation, Higgins et Sharp ont formalisé en 1989 la méthode CLUSTAL. Plutôt que d'optimiser l'alignement global simultanément, cette stratégie heuristique procède par itérations : les séquences les plus proches sont alignées en premier, puis les séquences plus distantes sont intégrées progressivement en suivant la topologie d'un arbre guide.

L'objectif de ce projet consiste à réimplémenter cette approche progressive en modifiant deux éléments de l'algorithme d'origine. Premièrement, la classification UPGMA (Unweighted Pair Group Method with Arithmetic mean, qui construit l'arbre en formant des sous-groupes indépendants) est remplacée par un algorithme d'embranchement séquentiel glouton. Cette méthode, plus directe, ajoute simplement les séquences une par une au profil existant. Deuxièmement, la méthode de Wilbur et Lipman (une technique d'alignement rapide mais approximative basée sur la recherche de mots) n'a pas été retenue. Le programme utilise à la place un alignement mathématiquement exact par programmation dynamique, associé au modèle de Gotoh (1982). Ce dernier applique des pénalités de gaps affines : il distingue le coût de création d'un espace de son coût de prolongement, ce qui reproduit fidèlement la réalité des mutations biologiques.

2. Matériel et Méthodes

2.1 Jeu de données et détection du type moléculaire

Le jeu de données utilisé pour ce projet rassemble dix séquences de la p53 (une protéine agissant comme suppresseur de tumeur) provenant de différents vertébrés. La séquence du cheval, téléchargée directement sur UniProt, présente la particularité d'être naturellement tronquée : elle ne compte que 280 acides aminés au lieu d'environ 390 pour les autres taxons. Nous l'avons volontairement gardée telle quelle dans le fichier d'entrée. Elle constitue un excellent test pratique pour observer comment l'algorithme gère un manque sévère d'information et force l'insertion de longs gaps aux extrémités.

Côté implémentation, le programme a été pensé pour être autonome. Dès la lecture du fichier FASTA, il vérifie l'intégralité des lettres pour identifier la nature des séquences. Si celles-ci se limitent à l'alphabet nucléique étendu (A, T, C, G, N, U, -), elles sont traitées comme de l'ADN. En revanche, la présence d'une seule lettre étrangère à cette liste suffit pour appliquer un traitement protéique. Cette flexibilité permet de charger la bonne matrice de substitution et les pénalités de gaps correspondantes sans que l'utilisateur n'ait à paramétrer l'outil manuellement.

2.2 Calcul des distances et pénalités de substitution

L'algorithme implémenté cherchant à minimiser un coût global d'alignement, les similarités biologiques doivent être converties en distances mathématiques. Deux résidus identiques génèrent ainsi une distance nulle, tandis que deux résidus chimiquement discordants induisent une distance maximale. Le barème appliqué dépend de la nature des molécules étudiées.

Pour les séquences nucléiques, nous reprenons le barème à trois niveaux défini par Higgins et Sharp (1989). L'identité stricte donne une distance de 0. Les transitions (mutations conservatrices entre deux purines ou deux pyrimidines), fréquentes sur le plan évolutif, reçoivent un coût modéré de 5. Les transversions, qui altèrent plus lourdement la structure cyclique des bases et sont plus rares, sont pénalisées par une distance de 10.

Pour les séquences protéiques, le programme exploite la matrice PAM250 de Dayhoff. Ses scores bruts de log-odds s'étendant de -8 à +17, un décalage de +8 est appliqué à la matrice brute lors de la lecture du fichier afin d'opérer sur des scores strictement positifs compris entre 0 et 25. La distance de substitution est ensuite évaluée par la formule $25 - \text{score}$. Ainsi, l'alignement de deux tryptophanes très conservés (score décalé de 25) produit une distance minimale de 0, tandis que l'appariement défavorable d'une cystéine et d'un tryptophane (score décalé de 0) reçoit la distance maximale de 25, ce qui est strictement fidèle aux calculs de l'article d'origine.

2.3 Alignement par paires : l'algorithme de Gotoh

L'alignement entre deux séquences s'effectue par programmation dynamique exacte. Biologiquement, l'insertion ou la suppression d'un bloc de plusieurs lettres consécutives découle le plus souvent d'un seul événement de mutation.

Appliquer une pénalité fixe à chaque lettre manquante viendrait donc fausser la réalité évolutive. Pour éviter ce biais, nous avons intégré le modèle de Gotoh (1982) et ses pénalités affines : le calcul sépare le coût d'ouverture d'un nouvel espace (lourdement pénalisé) de son prolongement (très peu pénalisé). Le choix de ces valeurs numériques doit rester proportionnel aux échelles de distances définies précédemment (0 à 10 pour l'ADN, 0 à 25 pour les protéines). Si les coûts de gaps étaient trop faibles par rapport aux coûts de substitution, l'algorithme minimisant la distance globale insérerait abusivement des espaces artificiels pour esquiver les mutations. Pour maintenir cet équilibre, le programme applique des pénalités de 10 (ouverture) et 2 (extension) pour l'ADN, contre 25 et 10 pour les protéines.

Au niveau de l'implémentation, l'algorithme remplit trois matrices en parallèle. La matrice M gère les correspondances directes, tandis que I_x et I_y gardent en mémoire l'état des gaps ouverts dans la première et la seconde séquence.

L'alignement final se reconstitue ensuite par *backtracking*, c'est-à-dire en remontant le chemin de coût minimum depuis la cellule terminale (située en bas à droite des matrices) jusqu'au point de départ. L'article de 1989 préconisait la méthode de Myers et Miller pour réduire l'espace mémoire nécessaire aux calculs. Nous avons plutôt fait le choix de conserver l'intégralité des trois matrices : sur les ordinateurs actuels, l'espace qu'elles occupent est négligeable pour les longueurs de séquences étudiées. La taille de ces matrices correspondant au produit des longueurs des deux séquences, cette implémentation impose une complexité spatiale quadratique en $O(L^2)$ (où L représente la longueur moyenne de nos séquences).

2.4 Matrice de similarité et arbre glouton

L'ordre d'alignement dépend d'une matrice de similarité, c'est-à-dire un tableau croisé qui stocke les scores d'affinité entre chaque paire de séquences possible. Puisque notre implémentation de Gotoh évalue une distance (qu'elle cherche à minimiser), il faut convertir ce résultat en un score de ressemblance (à maximiser) pour construire l'arbre. En suivant la méthode de l'article de 1989, ce score s'obtient en comptant le nombre d'identités strictes de la paire, auquel on soustrait le nombre de colonnes contenant au moins un gap. Afin d'économiser du temps de calcul, le programme n'évalue que la moitié supérieure de cette matrice symétrique, la diagonale restant nulle car comparer une séquence à elle-même est inutile.

Dans l'approche CLUSTAL classique, l'algorithme UPGMA utilise cette matrice pour former des sous-groupes hiérarchiques indépendants (des clades). Notre projet remplace cette étape par un embranchement séquentiel glouton. Ce changement modifie fondamentalement la structure de l'arbre : au lieu d'isoler des sous-groupes, l'arbre prend la forme d'un peigne où les séquences s'ajoutent simplement une par une au profil déjà formé.

En pratique, le programme repère d'abord la paire ayant la plus forte similarité absolue pour initialiser l'alignement. Ensuite, il parcourt les séquences restantes et intègre, tour à tour, celle qui présente le meilleur score avec l'une des séquences déjà incluses. L'alignement se construit donc séquence par séquence. La recherche de ce score maximum étant initialisée à moins l'infini, l'algorithme est sécurisé contre les scores négatifs générés par la soustraction des gaps chez des taxons très distants.

2.5 Alignement progressif et gestion des flottants

L'alignement multiple se construit séquence par séquence. Lorsqu'une nouvelle séquence est ajoutée à un groupe déjà aligné (un profil), les gaps qui s'y trouvent sont figés et conservés à l'identique.

Pour évaluer la distance entre la nouvelle séquence et une colonne de ce profil, l'article de 1989 propose d'utiliser une moyenne arithmétique. En informatique, diviser des entiers génère des valeurs à virgule (flottants) qui subissent des approximations lors de leur stockage en mémoire. Cela bloque techniquement le *backtracking* : pour retrouver son chemin, notre programme vérifie par une égalité stricte si le score actuel moins la distance correspond exactement à la valeur de la cellule précédente. Si les valeurs manipulées sont des flottants, ce test d'égalité échoue fréquemment à cause des arrondis de la machine.

Pour contourner ce problème, notre code remplace le calcul de la moyenne par la somme des distances de la colonne. Pour maintenir l'équilibre des scores, les pénalités d'ouverture et d'extension de gap sont simplement multipliées par le nombre de séquences du profil, noté K. Cette modification permet de réaliser l'intégralité de l'algorithme avec des nombres entiers, ce qui sécurise les tests d'égalité et garantit une reconstruction mathématiquement exacte de l'alignement.

2.6 Évaluation des performances algorithmiques

Afin d'évaluer l'efficacité de l'algorithme, le programme inclut deux tests de performance (benchmarks). Le premier analyse l'impact de la longueur des séquences (L) sur le temps d'exécution, en alignant des séquences variant de 50 à 300 résidus. Le second test mesure l'impact du nombre de séquences (N) : en conservant une longueur fixe de 50 résidus, le programme augmente simplement le nombre de séquences à aligner, en passant de 3 à 12. Pour identifier l'étape la plus coûteuse en temps de calcul, ce second test mesure séparément la durée de création de la matrice de similarité, de construction de l'arbre et de l'alignement progressif.

3. Résultats

3.1 Construction de l'arbre guide et ordre d'intégration

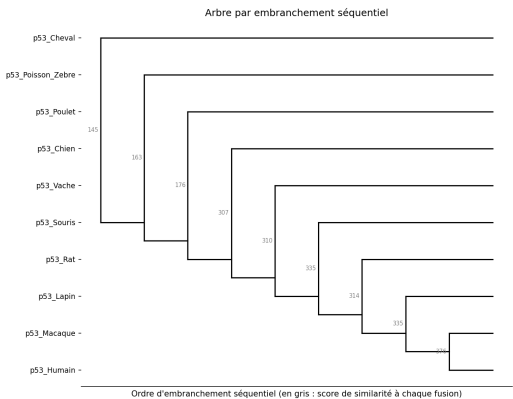
Dès la lecture du fichier FASTA contenant les séquences de la protéine p53, le programme a correctement identifié leur nature protéique, appliquant automatiquement le système de score adapté. Ensuite, il a calculé les similarités entre toutes les paires possibles. À partir de ce tableau, l'algorithme glouton a défini l'ordre dans lequel les séquences allaient être alignées, étape par étape (Tableau 1).

Tableau 1. Ordre d'intégration et scores de similarité

Étape	Séquence intégrée	Séquence de rattachement	Score de similarité
1 (Profil initial)	p53_Humain + p53_Macaque	-	376
2	p53_Lapin	p53_Humain ou p53_Macaque	335
3	p53_Rat	p53_Lapin	314
4	p53_Souris	p53_Rat	335
5	p53_Vache	p53_Lapin	310
6	p53_Chien	p53_Lapin	307
7	p53_Poulet	p53_Humain	176
8	p53_Poisson_Zebre	p53_Chien	163
9	p53_Cheval	p53_Chien	145

Le regroupement commence avec p53_Humain et p53_Macaque, car c'est la paire qui partage le score d'affinité le plus élevé (376). Cet ordre de construction est illustré sur la Figure 1 par un dendrogramme. Un dendrogramme est un diagramme sous forme d'arbre qui retrace visuellement la hiérarchie des regroupements. Sur ce graphique, la lecture se fait par la proximité des connexions : plus les séquences sont reliées tôt entre elles, plus elles sont similaires.

Figure 1. Dendrogramme de l'arbre guide.
Les valeurs grisées indiquent le score de similarité (identités moins les pénalités de gaps) ayant déclenché l'intégration de la séquence.



Ce schéma met également en évidence le comportement de l'approche gloutonne : on ne trouve pas de branches indépendantes qui évoluent séparément pour fusionner à la fin. L'arbre prend plutôt la forme d'un « peigne », chaque séquence venant s'ajouter une par une au profil déjà formé. L'assemblage s'achève avec la séquence p53_Cheval, qui rejoint le groupe en dernière position avec le score de similarité le plus bas (145).

3.2 Analyse de l'alignement multiple final

Le programme génère un alignement final d'une longueur totale de 411 colonnes. Concrètement, une colonne d'alignement correspond soit à un acide aminé, soit à un espace (un gap). La séquence la plus longue de notre jeu de données (celle de l'humain) compte initialement 393 acides aminés. Au fil des alignements progressifs, l'algorithme a inséré des espaces dans cette séquence pour pouvoir l'aligner face aux insertions présentes chez d'autres espèces (comme le poisson zèbre ou le poulet). La taille finale de 411 colonnes correspond ainsi à la longueur nécessaire pour faire correspondre l'ensemble des séquences.

Le Tableau 2 consigne la longueur d'origine de chaque séquence, le nombre exact d'espaces insérés par le programme, et leur proportion par rapport à cette taille totale de 411 colonnes.

Tableau 2. Statistiques d'alignement final (411 colonnes)

Séquence	Longueur d'origine	Gaps insérés	Proportion de gaps
p53_Humain	393	18	4.4 %
p53_Macaque	393	18	4.4 %
p53_Lapin	391	20	4.9 %
p53_Rat	391	20	4.9 %
p53_Souris	390	21	5.1 %
p53_Vache	386	25	6.1 %
p53_Chien	381	30	7.3 %
p53_Poisson_Zebre	373	38	9.2 %
p53_Poulet	367	44	10.7 %
p53_Cheval	280	131	31.9 %

Ces résultats chiffrés permettent de vérifier la précision mathématique du code, en particulier lors de la phase de *backtracking* (la remontée de la matrice). En prenant l'exemple de la séquence humaine, l'addition de ses 393 acides aminés de départ et des 18 espaces insérés donne exactement les 411 colonnes du résultat final. L'algorithme n'a donc généré aucune perte d'information lors des itérations.

Du côté des proportions, les séquences de taille standard (autour de 390 acides aminés) nécessitent peu d'ajustements : le macaque, très proche de l'humain, ne compte que 4,4 % de gaps. À l'inverse, des espèces plus distantes comme le poulet en requièrent davantage (10,7 %). Enfin, la séquence de la protéine p53 du cheval, initialement très courte (280 résidus), se démarque avec l'insertion de 131 espaces, soit 31,9 % de sa longueur alignée.

3.3 Mesures de performances (Benchmark)

Pour évaluer les limites du programme, un premier test s'intéresse à l'impact de la longueur des séquences (L) sur le temps d'exécution. En fixant le nombre de séquences à 5, nous avons fait varier leur longueur de 50 à 300 résidus (Tableau 3).

Tableau 3. Temps d'exécution en fonction de la longueur (N=5)

Longueur des séquences (L)	Temps de calcul total	Facteur d'accroissement
50 résidus	0.0345 s	-
100 résidus	0.1355 s	~ x 3.92
200 résidus	0.5493 s	~ x 4.05
300 résidus	1.2674 s	~ x 2.30

Les résultats montrent qu'à chaque doublement de la taille des séquences (de 50 à 100, puis à 200), le temps de calcul est multiplié par un facteur d'environ 4 (x3,92 et x4,05). Ce constat valide expérimentalement la complexité quadratique, notée $O(L^2)$, signifiant que le temps d'exécution croît proportionnellement au carré de la taille des données. En effet, l'algorithme de Gotoh remplit trois matrices (M, Ix, Iy) d'une surface totale de $L \times L$. Si l'on double la longueur L, cette surface de calcul devient $(2L)^2$, soit $4L^2$. L'ordinateur doit donc traiter une zone quatre fois plus vaste, ce qui justifie mécaniquement cette hausse stricte du temps machine.

Le second test (Tableau 4) maintient une longueur fixe de 50 résidus pour analyser l'impact du nombre de séquences (N) de 3 à 12, afin de repérer l'étape la plus coûteuse du pipeline.

Tableau 4. Décomposition du temps selon le nombre de séquences (L=50)

Nb de séquences (N)	Matrice	Arbre	Alignement progressif	Temps Total	Part Matrice
3 séquences	0.0065 s	0.0000 s	0.0055 s	0.0120 s	54.1 %
5 séquences	0.0219 s	0.0000 s	0.0126 s	0.0346 s	63.3 %
8 séquences	0.0610 s	0.0000 s	0.0283 s	0.0893 s	68.3 %
12 séquences	0.1431 s	0.0000 s	0.0544 s	0.1975 s	72.4 %

Les mesures révèlent que la construction de l'arbre glouton est quasi instantanée (0,0000 s). Cela s'explique par sa simplicité algorithmique : le programme cherche uniquement la valeur maximale dans le tableau des scores croisés, sans nécessiter de calcul mathématique complexe. À l'inverse, la création de la matrice de similarité initiale est l'étape la plus lourde. Elle impose mathématiquement d'effectuer $N \times (N - 1) / 2$ alignements de Gotoh complets pour comparer toutes les paires possibles. Plus le fichier contient de séquences, plus cette étape monopolise le temps global, passant de 54,1 % du temps pour 3 séquences à 72,4 % pour 12 séquences.

4. Discussion

L'utilisation des pénalités affines de Gotoh s'avère indispensable pour traiter les données issues de bases publiques comme UniProt. Plutôt que de disperser les espaces de façon aléatoire, ce modèle regroupe les insertions et délétions. Ce mécanisme a permis d'intégrer correctement la séquence du cheval, téléchargée nativement tronquée à 280 résidus, sans décaler l'alignement des autres espèces. Cette précision exige toutefois une occupation mémoire en $O(L_1 \times L_2)$. Si ce coût spatial reste mesuré pour une protéine comme la p53, il rendrait l'optimisation de Myers et Miller incontournable pour l'analyse de génomes complets.

Concernant la classification, l'arbre glouton s'éloigne de la méthode UPGMA en alignant chaque espèce successivement sur un profil unique au lieu de former des sous-groupes indépendants. Cet arbre met en évidence un artéfact méthodologique marquant : bien que le cheval soit un mammifère, sa troncature fait chuter son score de similarité globale, le reléguant en toute dernière position d'intégration (après les oiseaux et les poissons). Loin d'être un défaut de code, cette anomalie illustre l'impact direct d'une annotation incomplète sur la distance de Gotoh. À l'avenir, le remplacement de cette approche gloutonne par une classification de type Neighbor-Joining permettrait de calculer des distances évolutives globales et ainsi de stabiliser la topologie de l'arbre face à de telles séquences tronquées.

Enfin, la stabilité informatique du programme repose sur le rejet strict des nombres flottants. Le calcul classique d'un profil par des moyennes génère des erreurs d'arrondi qui compromettent l'étape de remontée de la matrice (backtracking). Le passage à une arithmétique entière, en multipliant simplement les pénalités d'espacement par le nombre de séquences (K), respecte le modèle mathématique originel tout en sécurisant définitivement l'exécution du code.

5. Conclusion

Ce projet aboutit à un outil d'alignement progressif fonctionnel, fidèle aux principes de CLUSTAL par l'utilisation des matrices PAM250 et la conservation stricte des espaces entre les itérations. Le modèle à pénalités affines gère efficacement les séquences incomplètes, l'arbre glouton accélère le regroupement initial, et les chronométrages valident la complexité en $O(L^2)$ prévue théoriquement. En définitive, ce travail illustre qu'un logiciel bioinformatique pertinent doit concilier la justesse de ses modèles biologiques avec les contraintes matérielles, comme le démontre le choix stratégique de l'arithmétique entière.

6. Références

Higgins, D.G. et Sharp, P.M. (1989) Fast and sensitive multiple sequence alignments on a microcomputer. *CABIOS*, 5(2), 151-153. Gotoh, O. (1982). An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3), 705-708. UniProt Consortium. (2021). UniProt: the universal protein knowledgebase in 2021. *Nucleic Acids Research*, 49(D1), D480-D489.